# IMPLEMENTAREA CONCURENTEI IN LIMBAJE DE PROGRAMARE

## CONCURENTA IN JAVA

Ioana Leustean

https://docs.oracle.com/javase/tutorial/essential/concurrency/

https://docs.oracle.com/javase/specs/jls/se23/jls23.pdf

Overview (Java SE 23 & JDK 23) (oracle.com)

Modele de interactiuni concurente:

- ✓ Dinning Philosophers
- ✓ Producer - Consumer
- ➢ Reader - Writer

## ➢Modelul de interactiune Cititori-Scriitori (Reader-Writers)

- Mai multe threaduri au acces la o resursa.
- Unele thread-uri scriu (writers), iar altele citesc (readers).
- Resursa poate fi accesata **simultan** de **mai multi cititori**.
- Resursa poate fi accesata de un **singur scriitor**.
- Resursa **nu** poate fi accesata simultan de cititori si de scriitori

## ➢ Interface ReadWriteLock

```
interface Lock
interface ReadWriteLock   extends Lock
class ReentrantReadWriteLock
```

- mentine o pereche de lacate: unul pentru citire si unul pentru scriere
- lacatul pentru citire poate fi detinut de mai multe thread-uri simultan, daca nu exista o solicitare pentru scriere
- lacatul pentru scriere poate fi detinut de un singur thread

- metoda **readLock()** intoarce lacatul pentru cititori
- metoda **writeLock()** intoarce lacatul pentru scriitori

ReadWriteLock (Java SE 23 & JDK 23) (oracle.com)

```
import java.util.concurrent.*;
import java.util.concurrent.locks.*;

public class ReaderWriter {
   private static Integer counter = 0;  // resursa

   private static ReadWriteLock lock = new ReentrantReadWriteLock();

   public static void main(String[] args) {

      (new Thread(new TaskW())).start();
      (new Thread(new TaskR())).start();
      (new Thread(new TaskR())).start();
      (new Thread(new TaskW())).start();
      (new Thread(new TaskR())).start();
      (new Thread(new TaskR())).start();
      (new Thread(new TaskR())).start();
      (new Thread(new TaskW())).start();
      }
```

Thread-ul **Writer**

```
private static class TaskW implements Runnable {
    public void run () {

      lock.writeLock().lock();

      try{
      int temp = counter;
      for (int i=0;i<5;i++) {counter++;          // fiecare thread Writer incrementeaza contorul de 5 ori
                                Thread.currentThread().sleep(1);}
      System.out.println(Thread.currentThread().getName() +
                                                  " Writer - before: "+temp+" after:" + counter);}
      catch (InterruptedException e){}
      finally {
       lock.writeLock().unlock();}
      }}
```

Thread-ul **Reader**

```
private static class TaskR implements Runnable {

    public void run () {

        lock.readLock().lock();

     try{
     System.out.println(Thread.currentThread().getName() + "Reader counter:" + counter);}

      finally { lock.readLock().unlock();}
}}}
```

- **Modelul de interactiune Cititori-Scriitori (Reader-Writers)**

```
Thread-0Writer - before: 0 after:5
Thread-2Reader: counter:5
Thread-1Reader: counter:5
Thread-3Writer - before: 5 after:10
Thread-4Reader: counter:10
Thread-5Reader: counter:10
Thread-7Writer - before: 10 after:15
Thread-6Reader: counter:15
```

```
Thread-0Writer - before: 0 after:5
Thread-2Reader: counter:5
Thread-1Reader: counter:5
Thread-3Writer - before: 5 after:10
Thread-4Reader: counter:10
Thread-5Reader: counter:10
Thread-6Reader: counter:10
Thread-7Writer - before: 10 after:15
```

## ➢ Semafor cu cantitate (quantity semaphore)

```
public class Semaphore
extends Object

Semaphore(int permits)  // constructor
```

- implementeaza un semafor cu cantitate
  care coordoneaza accesul la un numar precizat de resurse

- metoda **aquire()  / acquire(int permits)**
  thread-ul care apeleaza aquire cere accesul la o resursa;
  daca nu sunt resurse, thread-ul este blocat

- metoda **release() / release(int permits)**
  thread-ul care apeleaza release elibereaza accesul la o resursa

```
Semaphore sem = new Semaphore(n);

sem.acquire();

      ...  //sectiune critica


sem.release();
```

Semaphore (Java SE 23 & JDK 23) (oracle.com)

```
public class Semaphore
extends Object

Semaphore(int permits)  // constructor

Semaphore sem = new Semaphore(n);

sem.acquire();

      ...  //sectiune critica
sem.release();
```

Diferenta dintre un obiect construit cu **Semaphore(1)** si unul din clasa **Lock** este urmatoarea:

- lacatul intern al obiectului din clasa **Semaphore** este eliberat de orice thread care face **release**
- lacatul intern al obiectului din clasa **Lock** este eliberat numai de thread-ul care il detine

Varianta **Semaphore(int permits, true)** thread-urile care asteapta sa faca aquire sunt FIFO

https://docs.oracle.com/en/java/javase/23/docs/api/java.base/java/util/concurrent/Semaphore.html

Exemplu:

- un semafor coordoneaza accesul la 3 resurse
- exista 4 thread-uri care cer accesul la resursa
- dupa ce primeste accesul, fiecare thread executa 3 task-uri, apoi elibereaza resursa

```
public class Semaphores{

    static Semaphore semaphore = new Semaphore(3);

    static class MyThread extends Thread {

        // thread-ul va face aquire, va executa task-urile, apoi va face release
    }

public void main(String[] args) { ...}
}
```

```
public static void main(String[] args)   {
        MyThread t1 = new MyThread("A"); t1.start();
         .....
        MyThread t4 = new MyThread("D"); t4.start();
}
```

http://winterbe.com/posts/2015/04/30/java8-concurrency-tutorial-synchronized-locks-examples/

```
static class MyThread extends Thread {
      String name = "";
      MyThread(String name) { this.name = name;}

public void run()  {
  try {
        semaphore.acquire();
              try {

            for (int i = 1; i <= 3; i++) {
               System.out.println(name + " : is performing operation " + i }
            Thread.sleep(ThreadLocalRandom.current().nextInt(1000));}

                } finally { semaphore.release();}

   }  catch (InterruptedException e) {}
}}
```

**aquire** pune thread-urile in asteptare, deci poate arunca o exceptie

```
public class Semaphores{

  static Semaphore semaphore = new Semaphore(3);
  static class MyThread extends Thread {...}

public void main(String[] args) {
MyThread t1 = new MyThread("A"); t1.start();
MyThread t2 = new MyThread("B"); t2.start();
MyThread t3 = new MyThread("C"); t3.start();
MyThread t4 = new MyThread("D"); t4.start();
}}
```

```
A : is performing operation 1
B : is performing operation 1
C : is performing operation 1
B : is performing operation 2
C : is performing operation 2
B : is performing operation 3
A : is performing operation 2
D : is performing operation 1
D : is performing operation 2
C : is performing operation 3
A : is performing operation 3
D : is performing operation 3
```

```
static Semaphore semaphore = new Semaphore(3);

static class MyThread extends Thread {
      String name = "";
      MyThread(String name) { this.name = name;}

public void run()  {
  try {
        semaphore.acquire(2); // are nevoie de 2 resurse pentru executie
              try {...
        } finally { semaphore.release(1);}
   }  catch (InterruptedException e) {}
}}
public void main(String[] args) {
MyThread t1 = new MyThread("A"); t1.start();
MyThread t2 = new MyThread("B"); t2.start();
MyThread t3 = new MyThread("C"); t3.start();
MyThread t4 = new MyThread("D"); t4.start();
}
```

```
A : is performing operation 1
A : is performing operation 2
A : is performing operation 3
C : is performing operation 1
C : is performing operation 2
C : is performing operation 3
```

Thread-urile B si D nu au cum sa faca acquire pe 2 resurse

## ➢ Crearea obiectelor de tip Thread:

- Metoda directa
  - ca subclasa a clasei Thread
  - implementarea interfetei Runnable

```
public interface Runnable{
 public void run() ;
}

public class Thread
extends Object
implements Runnable
```

- Metoda abstracta
  - folosind interfata Executor

```
interface Executor

public interface ExecutorService
extends Executor

public class Executors
extends Object
```

➢ Framework-ul Executor

```
interface Executor
public interface ExecutorService
extends Executor

public class Executors
extends Object
```

- ExecutorService asigura crearea si managementul unei piscine de thread-uri.

```
ExecutorService  pool = Executors.newCachedThreadPool()

pool.execute( instanta Runnable )
```

Crearea thread-urilor

crerea unui piscine de thread-uri folosind o metoda a clasei **Executors** (care intoarce o instanta a interfetei **ExecutorService)**

https://download.java.net/java/early_access/valhalla/docs/api/java.base/java/util/concurrent/Executors.html

interface Executor
public interface ExecutorService extends Executor
public class Executors extends Object

➢ Metode ale clasei **Executors**:

- **newSingleThreadExecutor()**

"Creates an Executor that uses a single worker thread operating off an unbounded queue. (Note however that if this single thread terminates due to a failure during execution prior to shutdown, a new one will take its place if needed to execute subsequent tasks.)"

  - Un thread (normal) executa un singur task, dar un thread creat cu aceasta metoda poate executa secvential o serie de task-uri.

- **newCachedThreadPool()**

"Creates a thread pool that creates new threads as needed, but will reuse previously constructed threads when they are available. These pools will typically improve the performance of programs that execute many short-lived asynchronous tasks."

- **newFixedThreadPool(poolSize)**

"Creates a thread pool that reuses a fixed number of threads operating off a shared unbounded queue. At any point, at most n Threads threads will be active processing tasks. If additional tasks are submitted when all threads are active, they will wait in the queue until a thread is available."

```
interface Executor
public interface ExecutorService extends Executor
public class Executors extends Object
```

- Metode ale clasei **Executors**:
  - **newSingleThreadExecutor()**
  - **newCachedThreadPool()**
  - **newFixedThreadPool(poolSize)**

- Metode ale interfetei **ExecutorService**
  - **shutdown()**
    serviciul nu primeste task-uri noi, dar executa task-urile deja primite
  - **shutdownNow()**
    terminarea serviciului, fara a permite finalizarea executiilor
  - **awaitTermination(long timeout, TimeUnit unit)**
    pentru a permite finalizarea executiilor, impunand o limita temporara

## ➢ Metode sincronizate

- doua thread-uri care incrementeaza acelasi contor

```
public class Task  implements Runnable {
 static Integer counter = 0;

      public void run () {
         for (int i = 0; i < 5; i++) {
            performTask();
         }}

private synchronized void performTask () {
      int temp = counter;
      counter++;
      System.out.println(Thread.currentThread()
                              .getName() + " - before: "+temp+" after:" + counter);}
public static void main (String[] args) {.. }}
```

## ➢ Generarea thread-urilor folosind Executor

```
public static void main (String[] args) {
    Thread thread1 = new Thread(new Task());
    Thread thread2 = new Thread(new Task());
    thread1.start(); thread2.start();
    thread1.join(); thread2.join();  }
```

```
Thread-1 - before: 1 after:2
Thread-1 - before: 2 after:3
Thread-0 - before: 0 after:1
Thread-1 - before: 3 after:4
Thread-0 - before: 4 after:5
Thread-1 - before: 5 after:6
Thread-0 - before: 6 after:7
Thread-1 - before: 7 after:8
Thread-0 - before: 8 after:9
Thread-0 - before: 9 after:10
```

```
import java.util.concurrent.*;

public static void main (String[] args)  {

    ExecutorService  pool = Executors.newCachedThreadPool();
    for(int i=0;i<2;i++) {pool.execute(new Task());}
    pool.shutdown();
}
```

```
pool-1-thread-1 - before: 0 after:1
pool-1-thread-2 - before: 1 after:2
pool-1-thread-1 - before: 2 after:3
pool-1-thread-1 - before: 4 after:5
pool-1-thread-2 - before: 3 after:4
pool-1-thread-1 - before: 5 after:6
pool-1-thread-2 - before: 6 after:7
pool-1-thread-2 - before: 8 after:9
pool-1-thread-1 - before: 7 after:8
pool-1-thread-2 - before: 9 after:10
```

Thread-urile sunt numite  pool-1-thread-k

```
public static void main (String[] args)  {
    ExecutorService pool =
Executors.newFixedThreadPool(2);
    for(int i=0;i<3;i++) {pool.execute(new Task());}
    demo.shutdown();
}
```

in metoda run a clasei Task contorul e incrementat de 5 ori

```
pool-1-thread-2 - before: 1 after:2
pool-1-thread-1 - before: 2 after:3
pool-1-thread-2 - before: 3 after:4
pool-1-thread-1 - before: 4 after:5
pool-1-thread-2 - before: 5 after:6
pool-1-thread-1 - before: 6 after:7
pool-1-thread-2 - before: 7 after:8
pool-1-thread-1 - before: 8 after:9
pool-1-thread-2 - before: 9 after:10
pool-1-thread-1 - before: 10 after:11
pool-1-thread-1 - before: 11 after:12
pool-1-thread-1 - before: 12 after:13
pool-1-thread-1 - before: 13 after:14
pool-1-thread-1 - before: 14 after:15
```

sunt create 2 thread-uri, dar avem 3 task-uri, deci un thread executa 2 task-uri

## ➢ shutdown() cu awaitTermination()

```
import java.util.concurrent.*;

public static void main (String[] args)   throws InterruptedException {

      ExecutorService  pool = Executors.newCachedThreadPool();
      for(int i=0;i<2;i++) {pool.execute(new Task());}
      pool.shutdown();
       try {
          if (!pool.awaitTermination(3500, TimeUnit.MILLISECONDS)) {
                          pool.shutdownNow();  }
     } catch (InterruptedException e) { pool.shutdownNow();}
 }
```

```
pool-1-thread-1 - before: 0 after:1
pool-1-thread-2 - before: 1 after:2
pool-1-thread-1 - before: 2 after:3
pool-1-thread-1 - before: 4 after:5
pool-1-thread-2 - before: 3 after:4
pool-1-thread-1 - before: 5 after:6
pool-1-thread-2 - before: 6 after:7
pool-1-thread-2 - before: 8 after:9
pool-1-thread-1 - before: 7 after:8
pool-1-thread-2 - before: 9 after:10
```

Thread-urile sunt numite  pool-1-thread-k

**awaitTermination()**
Blocks until all tasks have completed execution after a shutdown request,
or the timeout occurs, or the current thread is interrupted, whichever happens first.

*Exemplu:* ReaderWriter - generarea thread-urilor folosind Executor

```
import java.util.concurrent.*;
import java.util.concurrent.locks.*;

public class ReaderWriterE{
   private static Integer counter = 0;
   private static final ReadWriteLock lock = new ReentrantReadWriteLock();

   public static void main (String[] args)  {
      ExecutorService pool = Executors.newCachedThreadPool();
      pool.execute(new TaskW());
      pool.execute(new TaskR());
      pool.execute(new TaskW());
      pool.execute(new TaskR());
      pool.execute(new TaskR());
      pool.shutdown();
   }
```

```
C:\Users\igleu\Documents\DIR\ICLP22\Curs 2022\Java2022\pg>java ReaderWriterE
pool-1-thread-1 - before: 0 after:5
pool-1-thread-6 counter:5
pool-1-thread-4 - before: 5 after:10
pool-1-thread-3 counter:10
pool-1-thread-2 counter:10
pool-1-thread-5 counter:10
pool-1-thread-7 - before: 10 after:15

C:\Users\igleu\Documents\DIR\ICLP22\Curs 2022\Java2022\pg>java ReaderWriterE
pool-1-thread-1 - before: 0 after:5
pool-1-thread-3 counter:5
pool-1-thread-4 - before: 5 after:10
pool-1-thread-2 counter:10
pool-1-thread-5 counter:10
pool-1-thread-7 - before: 10 after:15
pool-1-thread-6 counter:15
```

## ➢ Callable si Future

```
public interface Runnable {
   public void run();
}
```

**executa** un thread

```
public interface Callable <ResultType> {
   ResultType call() throws Exception;
}
```

**intoarce rezultatul executiei** unui thread

```
Callable<ResultType> callable = new Callable<ResultType>() {

public String call() throws Exception {
        // executie care dureaza
      return result;
   }};

ExecutorService exec=Executor.newSingleThreadExecutor
Future<ResultType>  future = exec.submit(callable)
```

un obiect **Callable** intoarce un obiect **Future**

https://www.callicoder.com/java-callable-and-future-tutorial/

- Callable si Future

```
Callable<String> callable = new Callable<String>() {

public String call() throws Exception {
      // Perform some computation
      Thread.sleep(2000);
      return "Return some result";
   }};

public static void main (String[] args) throws Exception{

ExecutorService exec=Executor.newSingleThreadExecutor();
Future<String>  future = exec.submit(callable) ;
…
}
```

**Callable** reprezinta o executie **asincrona**, al carei rezultat este recuperate cu ajutorul unui obiect **Future**

## ➢ Executie asincrona

- implementarea unei instante a clasei **Callable** care intoarce un <String>
- instanta va fi folosita pentru a crea un obiect **Future**

```
private static class TaskCallable implements Callable<String> {
      private static int ts;
      public  TaskCallable (int ts) {this.ts = ts;}

      public String call () throws InterruptedException {
         System.out.println("Entered Callable; sleep:"+ts);
         Thread.sleep(ts);
         return "Hello from Callable";
      }
  }
```

Callable reprezinta o executie **asincrona**, al carei rezultat este recuperate cu ajutorul unui obiect Future

```
ExecutorService executorService = Executors.newSingleThreadExecutor();
Future<String>  futureEx =executorService.submit(new   TaskCallable(time));
```

https://www.callicoder.com/java-callable-and-future-tutorial

- Future (rezultatul unei executii asincrone)

  - **ExecutorService.submit()** intoarce imediat, returnand un obiect Future. Din acest moment se pot executa diferite task-uri in parallel cu cea executata de obiectul Future.

  - Rezultatul returnat de obiectul Future este obtinut apeland **future.get()**.

  - Metoda **get()** a obiectelor Future va bloca thread-ul care o apeleaza pana cand se returneaza obiectului Future; daca task-ul executat este anulat sau thread-ul current este intrerupt, metoda **get()** arunca exceptii.

  - Metoda **isDone()** a obiectelor Future poate fi apelata pentru a vedea daca obiectul si-a terminat de executat task-ul.

```
import java.util.concurrent.*;
public class CallableFuture{

 public static void main (String[] args) throws Exception{

      ExecutorService pool = Executors.newSingleThreadExecutor();

      int time = ThreadLocalRandom.current().nextInt(1000, 5000);

      System.out.println("Creating the future");
      Future<String>  futureEx = pool.submit(new TaskCallable(time));

      System.out.println("Do something else while callable is getting executed");
      Thread.currentThread().sleep(time);

      System.out.println("Retrieve the result of the future");
      String result = futureEx.get();
      System.out.println(result);

      pool.shutdown();        }
```

https://www.callicoder.com/java-callable-and-future-tutorial/

```
public static void main (String[] args) throws Exception{
    ExecutorService pool = Executors.newSingleThreadExecutor();
    int time = ThreadLocalRandom.current().nextInt(1000, 5000);
    System.out.println("Creating the future");

    Future<String>  futureEx = pool.submit(new TaskCallable(time));
    System.out.println("Do something else while callable is getting executed");

    while(!futureEx.isDone()) {
      System.out.println("Task is still not done...");
      Thread.sleep(200);
    }
    System.out.println("Retrieve the result of the future");
    String result = futureEx.get();
    System.out.println(result);

    pool.shutdown();
  }
```

```java
public static void main (String[] args) throws Exception{
    ExecutorService pool = Executors.newSingleThreadExecutor();
    int time = ThreadLocalRandom.current().nextInt(1000, 5000);
    System.out.println("Creating the future");

    Future<String>  futureEx = pool.submit(new TaskCallable(time));
    System.out.println("Do something else while callable is getting
executed");

    while(!futureEx.isDone()) {
      System.out.println("Task is still not done...");
      Thread.sleep(200);
    }
    System.out.println("Retrieve the result of the future");
    String result = futureEx.get();
    System.out.println(result);

    pool.shutdown();
  }
```

```
Creating the future
Do something else while callable is getting executed
Task is still not done...
Entered Callable; sleep:1084
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Retrieve the result of the future
Hello from Callable
```

**Callable** reprezinta o executie **asincrona**, al carei rezultat este recuperate cu ajutorul unui obiect **Future**

https://docs.oracle.com/javase/tutorial/essential/concurrency

```
public static void main (String[] args) throws Exception{
    ExecutorService pool = Executors.newSingleThreadExecutor();
    int time = ThreadLocalRandom.current().nextInt(1000, 5000);
    System.out.println("Creating the future");

    Future<String>  futureEx = pool.submit(  ()->{  try  {Thread.sleep(time);}
                                                    catch (InterruptedException e) {System.out.println("error");}
                                                     finally {return "Hello from Callable!";}}
                                                );

    System.out.println("Do something else while callable is getting executed");

    while(!futureEx.isDone()) {
       System.out.println("Task is still not done...");
       Thread.sleep(200);
    }
    System.out.println("Retrieve the result of the future");
    String result = futureEx.get();
    System.out.println(result);

    pool.shutdown();
  }
```

Java 8
argumentul lui submit este o functie anonima

- CompletableFuture

```
public static void main (String[] args) throws Exception{
    ExecutorService pool = Executors.newSingleThreadExecutor();
    int time = ThreadLocalRandom.current().nextInt(1000, 5000);
    System.out.println("Creating the future");

    Future<String>  futureEx = pool.submit(  ()->{  try  {Thread.sleep(time);}
                                                    catch (InterruptedException e) {System.out.println("error");}
                                                     finally {return "Hello from Callable!";}} );
   CompletableFuture.runAsync(()->{
            try {Thread.sleep(1000);} catch (InterruptedException e){System.out.println("error");}
            finally {System.out.println("Hello from Completable Future!");}});

    System.out.println("Do something else while callable is getting executed");

    while(!futureEx.isDone()) {
      System.out.println("Task is still not done..."); Thread.sleep(200);}
    System.out.println("Retrieve the result of the future");
    String result = futureEx.get();
    System.out.println(result);
    pool.shutdown();   }
```

- CompletableFuture implementeaza interfata Future si reprezinta o executie asincrona

```
public static void main (String[] args) throws Exception{
    ExecutorService pool = Executors.newSingleThreadExecutor();
    int time = ThreadLocalRandom.current().nextInt(1000, 5000);
    System.out.println("Creating the future");

    Future<String>  futureEx = pool.submit(  ()->{  try  {Thread.sleep(time);}
                                                    catch (InterruptedExceptio
{System.out.println("error");}
                                                    finally {return "Hello from
    CompletableFuture.runAsync(()->{
            try {Thread.sleep(1000);} catch (InterruptedException e){Syst
            finally {System.out.println("Hello from Completable Future!"

    System.out.println("Do something else while callable is getting execute

    while(!futureEx.isDone()) {
      System.out.println("Task is still not done..."); Thread.sleep(200);}
    System.out.println("Retrieve the result of the future");
    String result = futureEx.get();
    System.out.println(result);
    pool.shutdown();   }
```

```
Creating the future
Do something else while callable is getting executed
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Hello from Completable Future!
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Retrieve the result of the future
Hello from Callable!
```

- CompletableFuture reprezinta o executie asincrona **care poate fi completata**

```
public static void main (String[] args) throws Exception{
    ExecutorService pool = Executors.newSingleThreadExecutor();
    int time = ThreadLocalRandom.current().nextInt(1000, 5000);
    System.out.println("Creating the future");

    Future<String>  futureEx = pool.submit(  ()->{  try  {Thread.sleep(time);}
                                                    catch (InterruptedException e) {System.out.println("error");}
                                                     finally {return "Hello from Callable!";}} );
   CompletableFuture.runAsync(()->{
            try {Thread.sleep(1000);} catch (InterruptedException e){System.out.println("error");}
            finally {return "Hello from Completable Future!");}}. thenAccept((String s)->
                                                                    {System.out.println(s);});

    System.out.println("Do something else while callable is getting executed");

    while(!futureEx.isDone()) {
      System.out.println("Task is still not done..."); Thread.sleep(200);}
    System.out.println("Retrieve the result of the future");
    String result = futureEx.get();
    System.out.println(result);
    pool.shutdown();   }
```

https://docs.oracle.com/javase/tutorial/essential/concurrency

- CompletableFuture reprezinta o executie asincrona **care poate fi completata**

```
public static void main (String[] args) throws Exception{
    ExecutorService pool = Executors.newSingleThreadExecutor();
    int time = ThreadLocalRandom.current().nextInt(1000, 5000);
    System.out.println("Creating the future");

    Future<String>  futureEx = pool.submit(  ()->{  try  {Thread.sleep(time);}
                                                    catch (InterruptedException e)
{System.out.println("error");}
                                                    finally {return "Hello from Calla
 CompletableFuture.runAsync(()->{
        try {Thread.sleep(1000);} catch (InterruptedException e){System.out.prin
         finally {return "Hello from Completable Future!");}}. thenAccept(
                                                (String s)->{System.out.println(s);});

    System.out.println("Do something else while callable is getting executed");

    while(!futureEx.isDone()) {
       System.out.println("Task is still not done..."); Thread.sleep(200);}
    System.out.println("Retrieve the result of the future");
    String result = futureEx.get();
    System.out.println(result);
    pool.shutdown();   }
```

```
Creating the future
Do something else while callable is getting executed
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Hello from Completable Future!
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
Retrieve the result of the future
Hello from Callable!
```

thread-ul principal nu este blocat asteptand rezultatul

https://docs.oracle.com/javase/tutorial/essential/concurrency

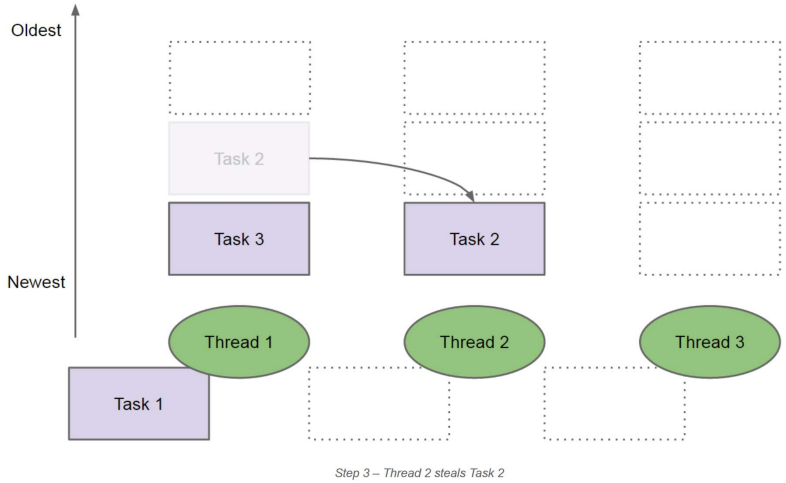
- CompletableFuture reprezinta o executie asincrona **care poate fi completata: thenApply(), thenAccept()**

```
public static void main (String[] args) throws Exception{
    ExecutorService pool = Executors.newSingleThreadExecutor();
    int time = ThreadLocalRandom.current().nextInt(1000, 5000);
    System.out.println("Creating the future");

    Future<String>  futureEx = pool.submit(  ()->{  try  {Thread.sleep(time);}
                                                    catch (InterruptedException e) {System.out.println("error");}
                                                     finally {return "Hello from Callable!";}} );
 CompletableFuture.runAsync(()->{
        try {Thread.sleep(1000);} catch (InterruptedException e)  {System.out.println("error");}
         finally {return "Hello from Completable Future!");}}. thenApply(
                                                                    (String s)-> s+"!"). thenAccept(
                                                                          (String s)->{System.out.println(s);});

    System.out.println("Do something else while callable is getting executed");

    while(!futureEx.isDone()) {
       System.out.println("Task is still not done..."); Thread.sleep(200);}
    System.out.println("Retrieve the result of the future");
    String result = futureEx.get();
    System.out.println(result);
    pool.shutdown();   }
```

rezultatul a fost transformat
cu ajutorul unei functii,
iar noul rezultat a fost prelucrat

```
Task is still not done...
Task is still not done...
Task is still not done...
Hello from Completable Future!!
Task is still not done...
Task is still not done...
Task is still not done...
Task is still not done...
```

## ➢ Fork-Join Framework




http://www.javacreed.com/java-fork-join-example/

A ForkJoinPool differs from other kinds of ExecutorService mainly by virtue of employing *work-stealing*: all threads in the pool attempt to find and execute tasks submitted to the pool and/or created by other active tasks (eventually blocking waiting for work if none exist).
ForkJoinPool (Java SE 23 & JDK 23)

## ➢ Fork-Join Framework

```
public class ForkJoinPool
extends AbstractExecutorService
```

```
class AbstractExecutorService
extends Object
implements ExecutorService
```

Diferenta dintre o piscine din clasa **ForkJoinPool** si cele create de alte servicii **Executor** este implementarea unei metode de "work-stealing".

Java 9 Concurrency Cookbook

The core of the *fork/join framework* is formed by the following two classes:

- **ForkJoinPool:** This class implements the ExecutorService interface and the work-stealing algorithm. It manages the worker threads and offers information about the status of the tasks and their execution.

- **ForkJoinTask:** This is the base class of the tasks that will execute in the ForkJoinPool. It provides the mechanisms to execute the fork() and join() operations inside a task and the methods to control the status of the tasks. Usually, to implement your fork/join tasks, you will implement a subclass of three subclasses of this class:

  **RecursiveAction** for tasks with no return result,
  **RecursiveTask** for tasks that return one result.

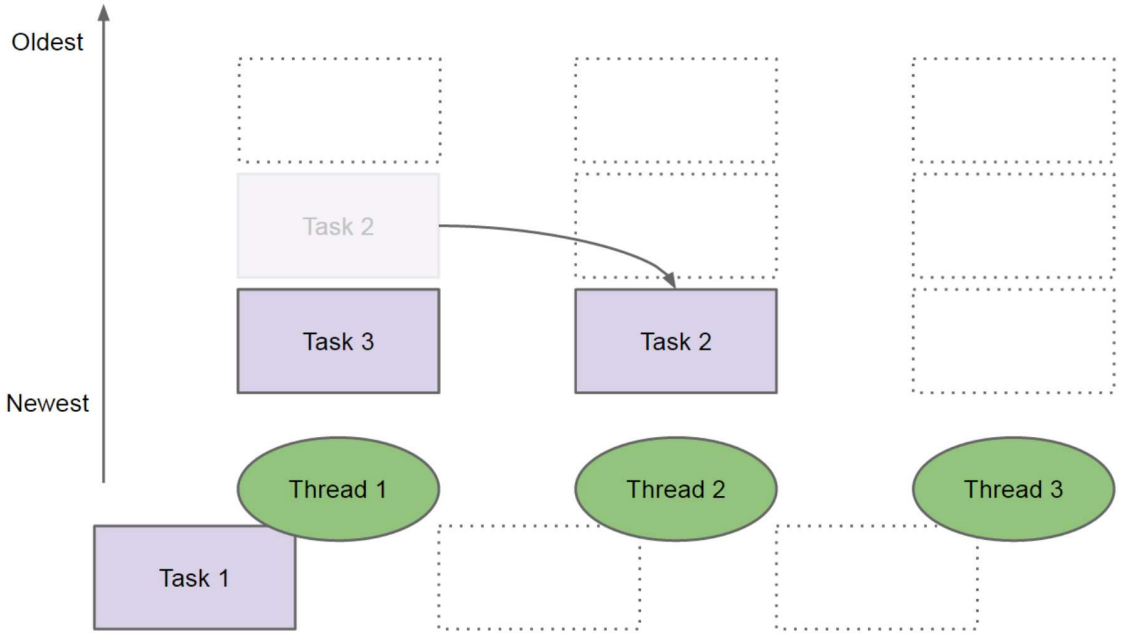
Documentatie:
[ForkJoinPool (Java SE 23 & JDK 23)]
https://www.researchgate.net/publication/2609854_A_Java_ForkJoin_Framework

*Daca*

- Thread1 executa Task1
- Task1 creaza Task2 si Task3 si
  are nevoie de rezultatele lor pentru a continua

*atunci*

- Thread1 pune in asteptare Task1 si va pune in coada proprie Task2 si Task3
- Thread1 va fi liber pentru a executa un task, celalalt va fi furat de un alt thread liber




*Step 3 – Thread 2 steals Task 2*

http://www.javacreed.com/java-fork-join-example/

- **Fork-Join Framework**

```
public class ForkJoinPool
extends AbstractExecutorService
```

- Crerea piscinei de threaduri
- Crearea task-ului
- Task-ul este trimis piscinei folosind

**invoke()** - semantic echivalenta cu fork();join()
- trimite task-ul in executie si intoarce rezultatul
- executia incepe in thread-ul current

**invokeAll() -** trimite spre executie o lista de task-uri si intoarce cand toate si-au terminat executia

- Crearea piscinei de thread-uri

```
ForkJoinPool fjpool = ForkJoinPool.commonPool() // recomandat,
                                                // incearca sa foloseasca toate procesoarele disponibile
ForkJoinPool fjpool =  new ForkJoinPool() // new ForkJoinPool(5)
```

- Crerea task-urilor

```
public abstract class ForkJoinTask<V>
extends Object
implements Future<V>
```

```
public abstract class RecursiveAction
extends ForkJoinTask<Void>
```

```
public abstract class RecursiveTask<V>
extends ForkJoinTask<V>
```

metoda compute implementeaza actiunea executata de task

- **Fork-Join Framework: ForkJoinTask<V>**
-

```
public class ForkJoinRecAc extends RecursiveAction {

public ForkJoinRecAc (long workLoad) {
      this.workLoad = workLoad;
   }

protected void compute() {
......
}}

public static void main (String[] args){
ForkJoinPool fjpool =  ForkJoinPool.commonPool(4)
ForkJoinRecAc fjaction= new ForkJoinRecAc(workLoad);
fjpool.invoke(fjaction);}
```

```
public class ForkJoinRecTk  extends RecursiveTask<T> {

public ForkJoinRecTk (long workLoad) {
      this.workLoad = workLoad;
   }

protected <T> compute() {
......
}}

public static void main (String[] args){
ForkJoinPool fjpool =  ForkJoinPool.commonPool(4)
ForkJoinRecTk  fjtask= new ForkJoinRecTk(workLoad);
<T> result = fjpool.invoke(fjtask);}
```

In exemple folosim:
http://tutorials.jenkov.com/java-util-concurrent/java-fork-and-join-forkjoinpool.html
http://www.baeldung.com/java-fork-join

- **Fork-Join Framework** cu **RecursiveAction** (forma generala)

```
public class MyRecursiveAction extends RecursiveAction {

public MyRecursiveAction (long workLoad) {
      this.workLoad = workLoad;
   }
protected void compute() {

 if (this.workLoad > limit) {…
                          List<MyRecursiveAction> subtasks = new ArrayList<MyRecursiveAction>();
                           subtasks.addAll(createSubtasks());
                           invokeAll(subtasks);}
                         }
else {// prelucrata de thread-ul curent}

}
private List<MyRecursiveAction> createSubtasks() {
      List<MyRecursiveAction> subtasks =  new ArrayList<MyRecursiveAction>();
…. }
```

invokeAll(Collection<T> tasks)

Trimite in executie toate task-urile (face fork() pe toate task-urile)

Exemplu: **crearea subtask-urilor**

```
private List<MyRecursiveAction>  createSubtasks() {

    List<MyRecursiveAction> subtasks =  new ArrayList<MyRecursiveAction>();

    MyRecursiveAction subtask1 = new MyRecursiveAction(this.workLoad / 2);
    MyRecursiveAction subtask2 = new MyRecursiveAction(this.workLoad / 2);

    subtasks.add(subtask1);
    subtasks.add(subtask2);

    return subtasks;
  }
```

Exemplu: **Fork-Join Framework cu RecursiveAction**

```
protected void compute() {

if  (this.workLoad > 15) {

        System.out.println("Splitting workLoad : " + this.workLoad);

        List<MyRecursiveAction> subtasks = new ArrayList<MyRecursiveAction>();

        subtasks.addAll(createSubtasks()); invokeAll(subtasks);}


else {
        System.out.println("Doing workLoad myself: " + this.workLoad);
    }
  }
```

Exemplu: **Fork-Join Framework** cu **RecursiveAction**

```
public static void main (String[] args){

    ForkJoinPool forkJoinPool = new ForkJoinPool(4);

    MyRecursiveAction myRecursiveAction = new MyRecursiveAction(64);

    forkJoinPool.invoke(myRecursiveAction);
  }}
```

```
Splitting workLoad : 64
Splitting workLoad : 32
Splitting workLoad : 32
Splitting workLoad : 16
Splitting workLoad : 16
Doing workLoad myself: 8
Splitting workLoad : 16
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Splitting workLoad : 16
Doing workLoad myself: 8
Doing workLoad myself: 8
```

- **Fork-Join Framework cu RecursiveTask <T>** (forma generala)

```
public class MyRecursiveTask extends RecursiveTask <T> {

public MyRecursiveTask (long workLoad) {
      this.workLoad = workLoad;
   }
protected  T compute() {

 if (this.workLoad > limit) {.....
                          List<MyRecursiveTask> subtasks = new ArrayList<MyRecursiveTask>();
                           subtasks.addAll(createSubtasks());
                           invokeAll(subtasks);
                           return joinresult(subtasks);}

else {// prelucrata de thread-ul current;
         return result}}

private List<MyRecursiveTask> createSubtasks() {
      List<MyRecursiveTask> subtasks =  new ArrayList<MyRecursiveTask>();    .... }
```

*joinresult(subtasks)* calculeaza rezultatul rezultatele subtaskurilor se obtin cu **subtask.get()**

https://docs.oracle.com/javase/tutorial/essential/concurrency

Exemplu: **Fork-Join Framework** cu **RecursiveTask<V>**

```
protected Integer compute() {
if(this.workLoad > 15) {
      System.out.println("Splitting workLoad : " + this.workLoad);
      List<MyRecursiveTask> subtasks = new ArrayList<MyRecursiveTask>();
      subtasks.addAll(createSubtasks());
      invokeAll(subtasks);

      int result = 0;
         try{
         for(MyRecursiveTask subtask : subtasks) {  result = result +  2* subtask.get();}
         System.out.println("Partial result: " + result);
         } catch (InterruptedException | ExecutionException e) {};
         return result;   }

else {System.out.println("Doing workLoad myself: " + this.workLoad);
         return workLoad;}
}
```

*joinresult(subtasks)* calculeaza rezultatul
rezultatele subtaskurilor se obtin cu subtask.get()

https://docs.oracle.com/javase/tutorial/essential/concurrency

Exemplu: **Fork-Join Framework** pool cu **RecursiveTask**

```
Splitting workLoad : 64
Splitting workLoad : 32
Splitting workLoad : 32
Splitting workLoad : 16
Splitting workLoad : 16
Splitting workLoad : 16
Doing workLoad myself: 8
Splitting workLoad : 16
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Partial result: 32
Doing workLoad myself: 8
Partial result: 32
Partial result: 32
Partial result: 32
Partial result: 128
Partial result: 128
Partial result: 512
Result= 512
```

```
public static void main (String[] args){

    ForkJoinPool forkJoinPool = ForkJoinPool.commonPool();

    MyRecursiveTask myRecursiveTask = new MyRecursiveTask(64);

    int res = forkJoinPool.invoke(myRecursiveTask);

    System.out.println("Result= " + res);}
```

Exemplu program: ForkJoin pool cu **RecursiveTask**

```
Splitting workLoad : 64
Splitting workLoad : 32
Splitting workLoad : 16
Splitting workLoad : 32
Doing workLoad myself: 8
Splitting workLoad : 16
Splitting workLoad : 16
Splitting workLoad : 16
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Partial result: 32
Doing workLoad myself: 8
Partial result: 32
Doing workLoad myself: 8
Partial result: 32
Partial result: 32
Partial result: 128
Partial result: 128
Partial result: 512
Result= 512
```

ForkJoin pool cu **RecursiveAction**

```
Splitting workLoad : 64
Splitting workLoad : 32
Splitting workLoad : 32
Splitting workLoad : 16
Splitting workLoad : 16
Doing workLoad myself: 8
Splitting workLoad : 16
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Doing workLoad myself: 8
Splitting workLoad : 16
Doing workLoad myself: 8
Doing workLoad myself: 8
```